Neural Network
Back Propagation Training Algorithm

CSAI 253 - Spring 2024

Prepared By

Dr. Khaled Mostafa ElSayed

kmelsayed@zewailcity.edu.eg

Instructors:

Dr. Khaled Mostafa ElSayed

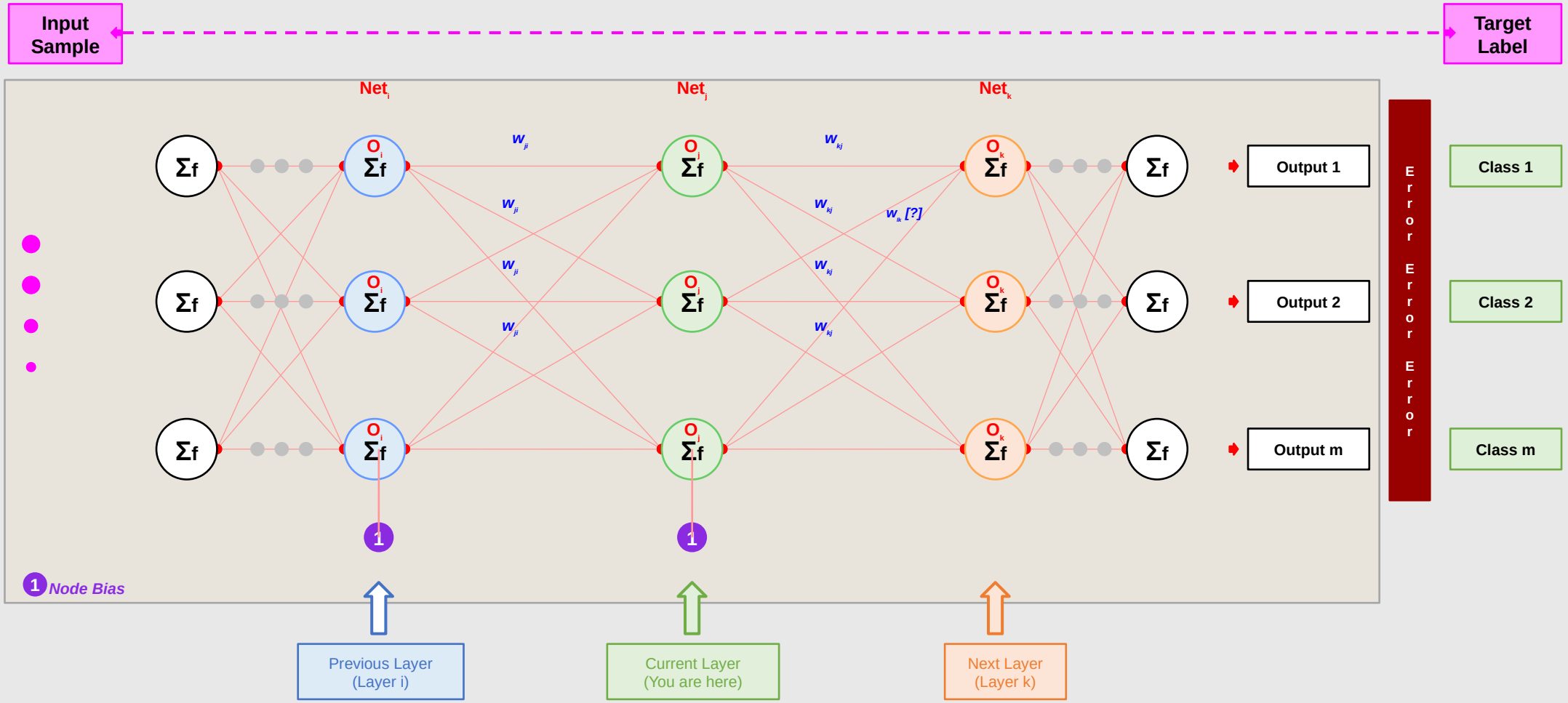
kmelsayed@zewailcity.edu.eg

Dr. Mohamed Maher Ata

momaher@zewailcity.edu.eg

Neural Network Model

(Each node consists of Summation + Activation)

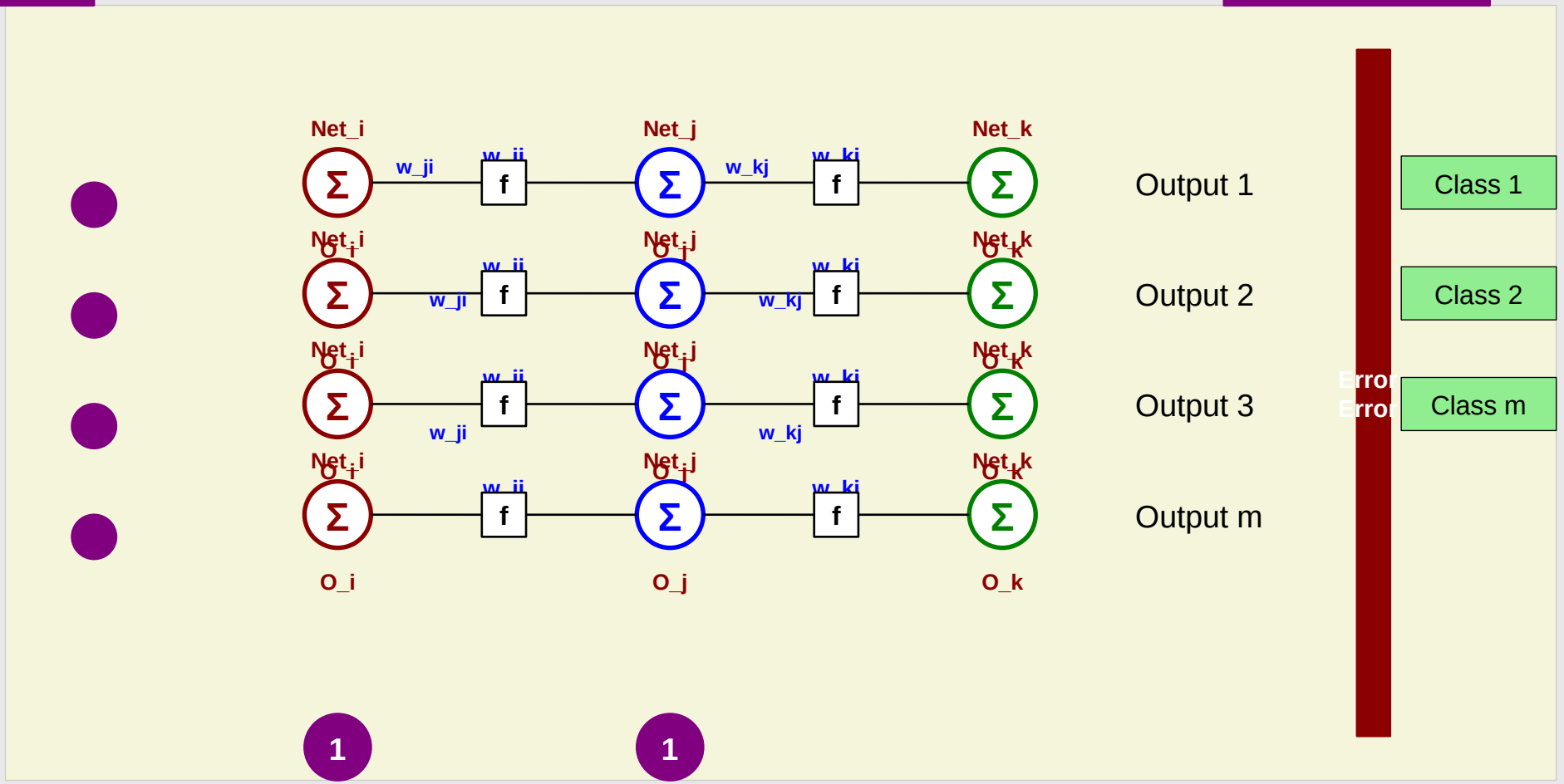


Neural Network Model

(Each node consists of Summation + Activation)

Input Sample

Target Label



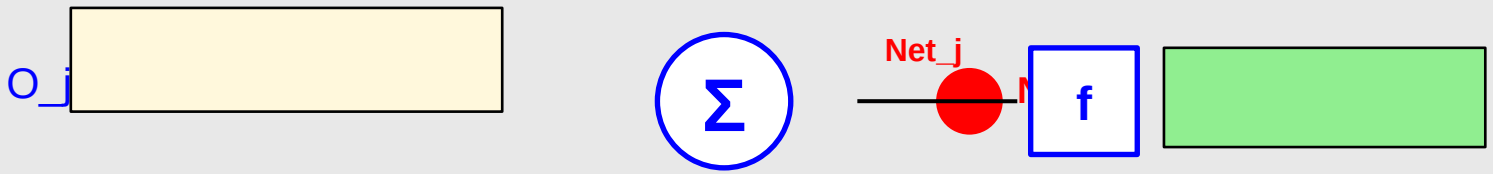
For Layer "j"

Previous Layer
Layer (i)

↑ **i**

Current Layer is Output L
(You are here)

Remember: Derivative of Sigmoid Function



Revision

Chain Rule (*if a is changed , c will be changed accordingly)*

Conclusion: b=f(W)= then *One Term ONLY*

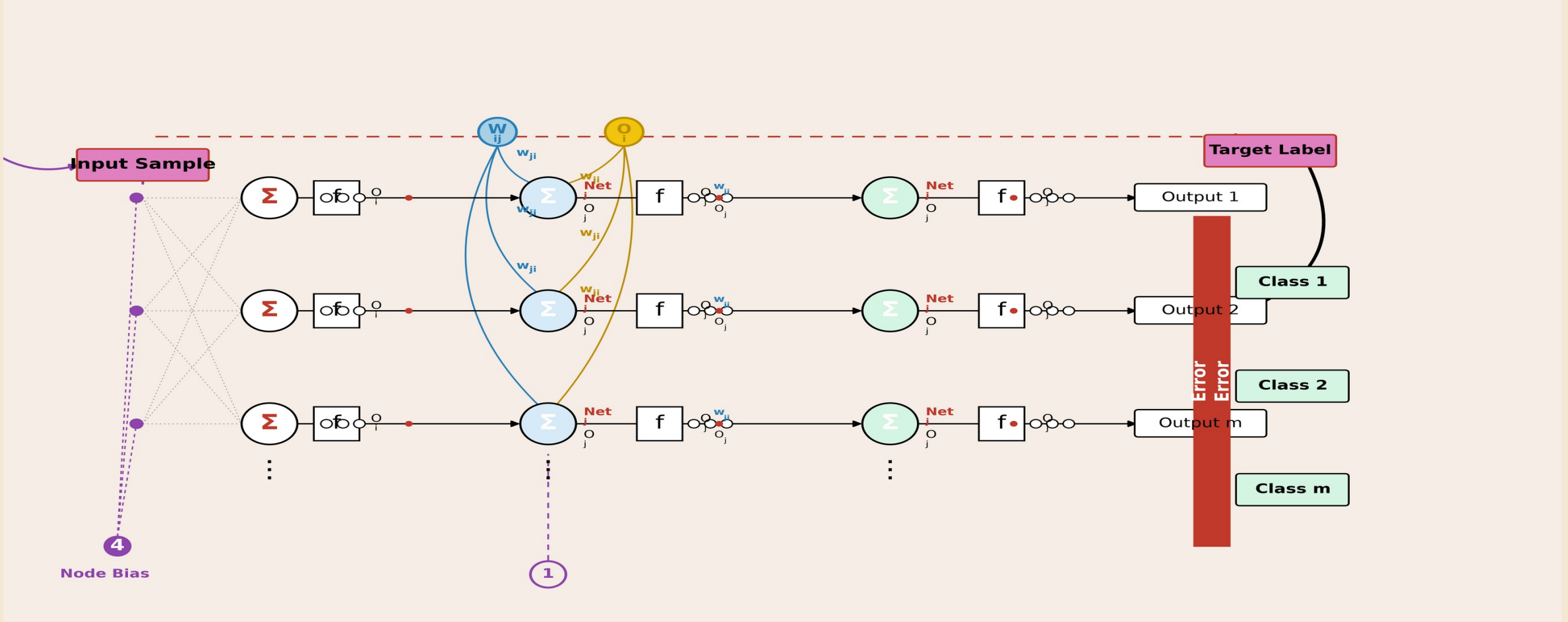
and

Update Weights of Output Layer

(Weights from previous layer to output layer)

Neural Network Model

(Weights of Output Layer AFFECTS Calculated Error)



For Layer "j"

$$Net_j = \sum w_{ji} O_i$$

$$O_j = f(Net_j) = \frac{1}{1 + e^{-Net_j}}$$

↑

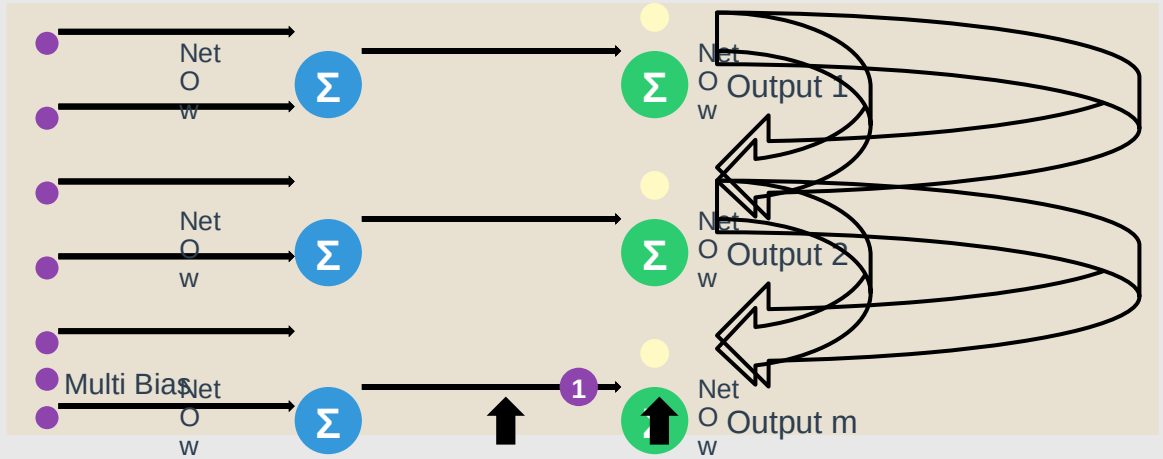
Previous Layer Layer (0)

↑

Current Layer is Output Layer (You are here)

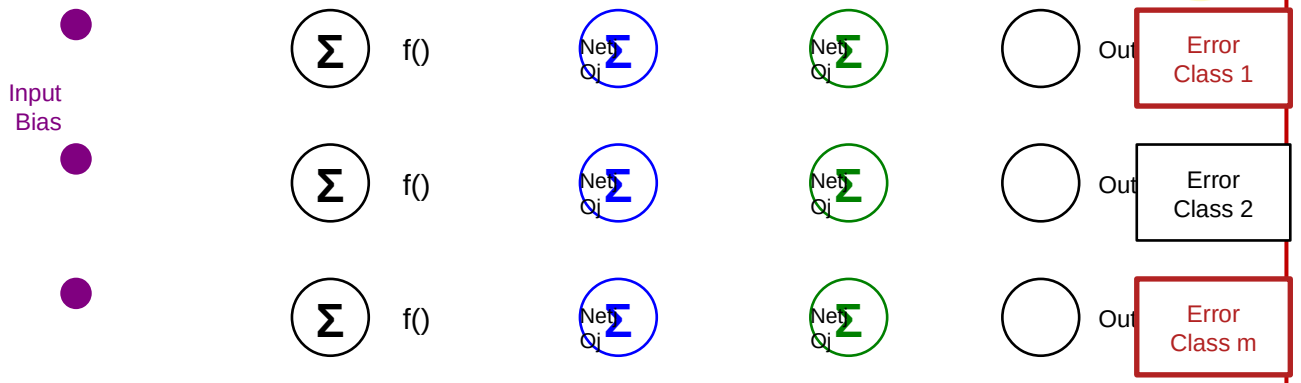
If Layer "j" is Output Layer:

$$E = \frac{1}{2} \sum (t_j - O_j)^2$$


Net
O
w
Σ
Net
O
w
Output 1
Net
O
w
Σ
Net
O
w
Output 2
Multi Bias
1
Net
O
w
Σ
Net
O
w
Output m
Error Class 1
Error Class 2
Error Class m
Error
?

Gradient Descent

Gradient Descent

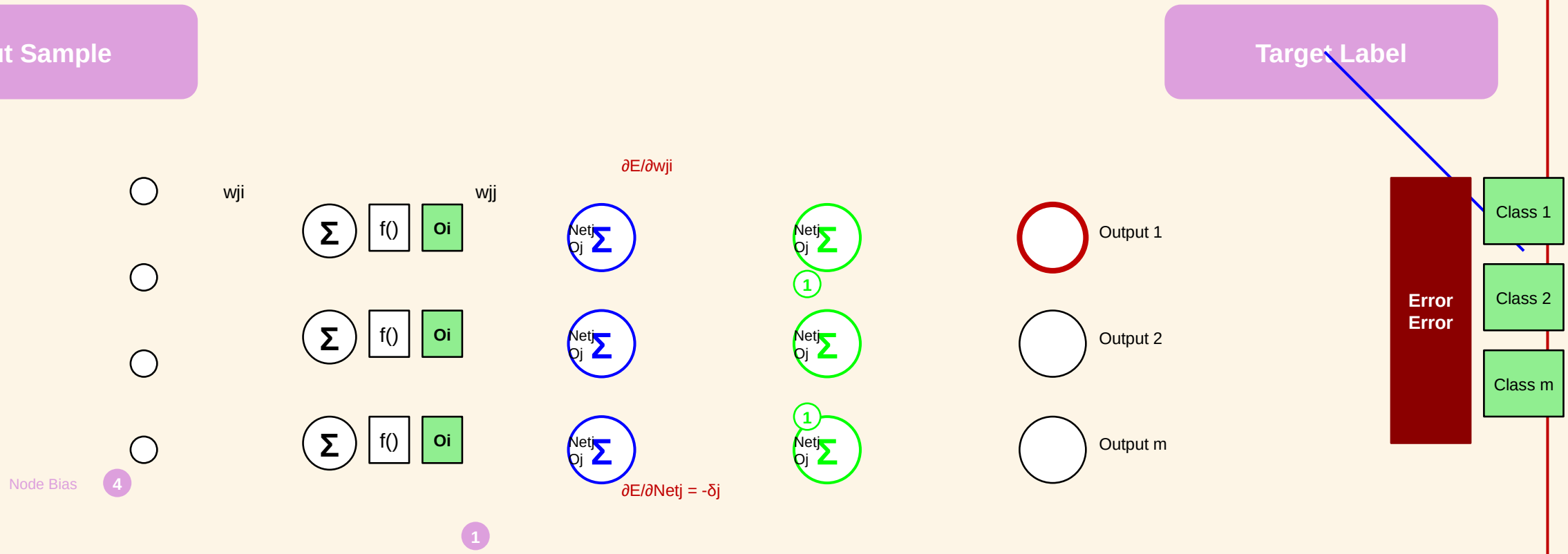


For Sigmoid Activation

Where δj = (tj - Oj) * Oj(1-Oj) for output Layer

Updated Weights of Output Layer

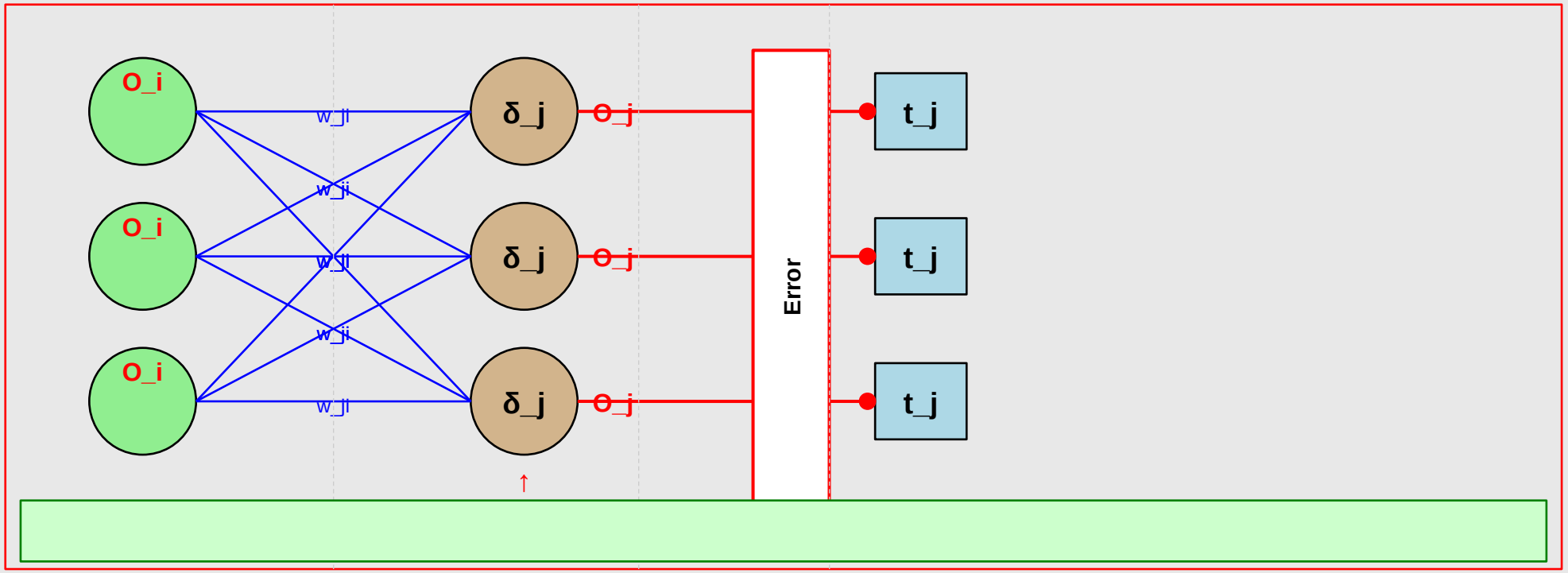
(Calculated Error is affected by Netj,)



Where δj = -∂E/∂Netj (one δ for each Node j)

Sigmoid

At Output Layer



Where:

Update Weights of Hidden Layer

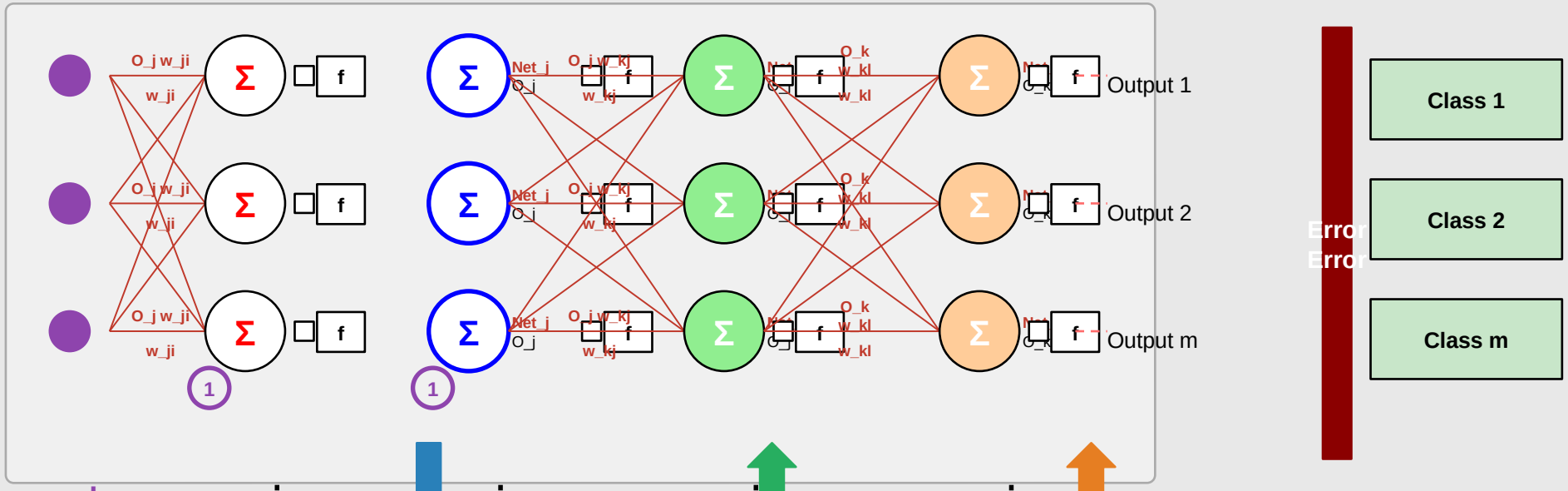
(Weights from previous layer to Hidden layer)

Neural Network Model

(Current Layer is Hidden Layer)

Input Sample

Target Label



For Layer "j"

Previous Layer
(I)

Current Layer
(You are here)

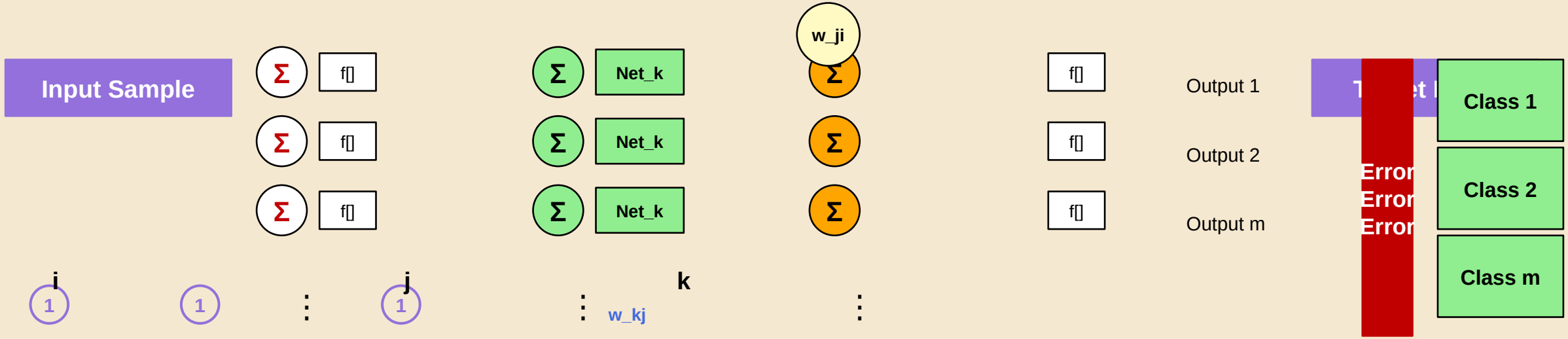
Next Layer
(K)

Note: Layer "j" is **NOT** Output Layer

X X X

Neural Network Model

(Effect of Net_k on Calculated Error is known from O/P Layer)



For Layer "j"

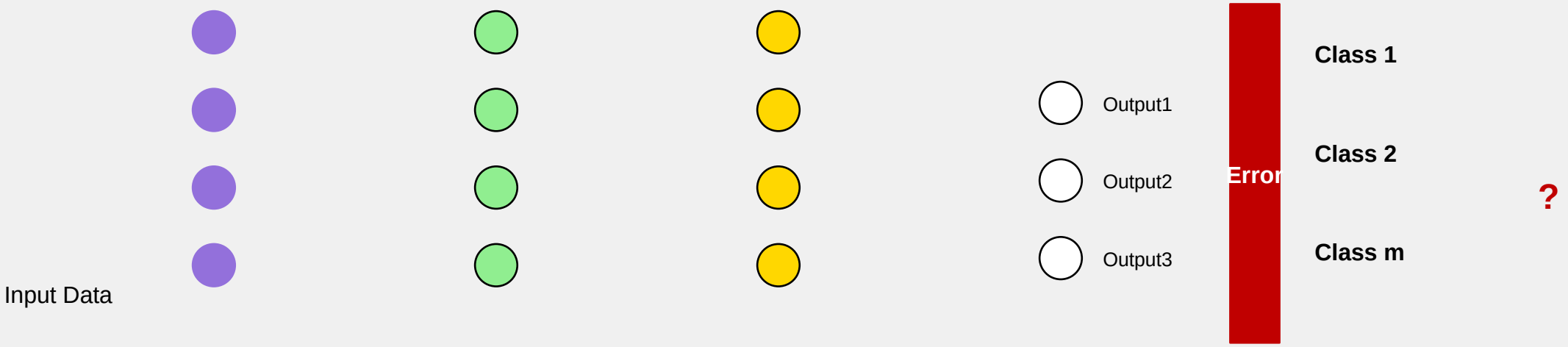
↑

Previous Layer (l)

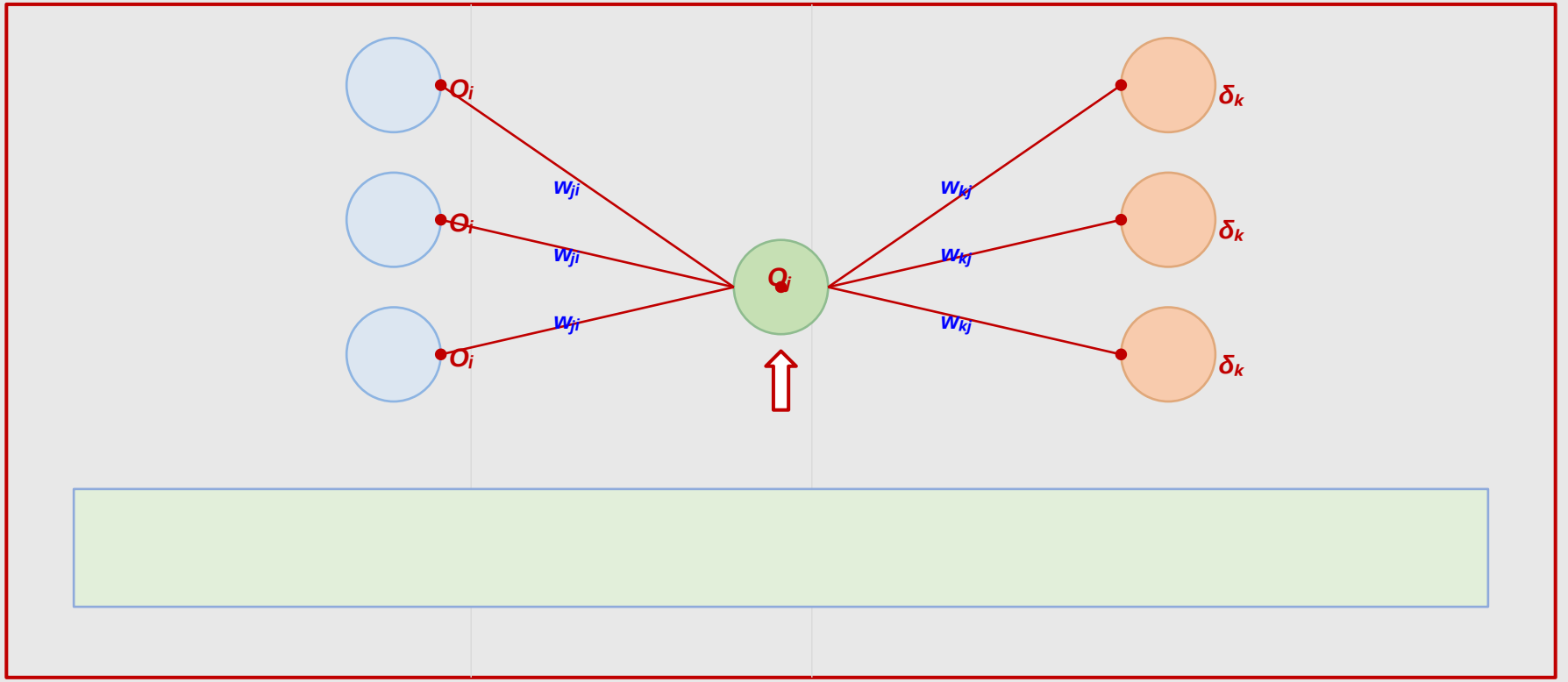
↑

Current Layer (You are h

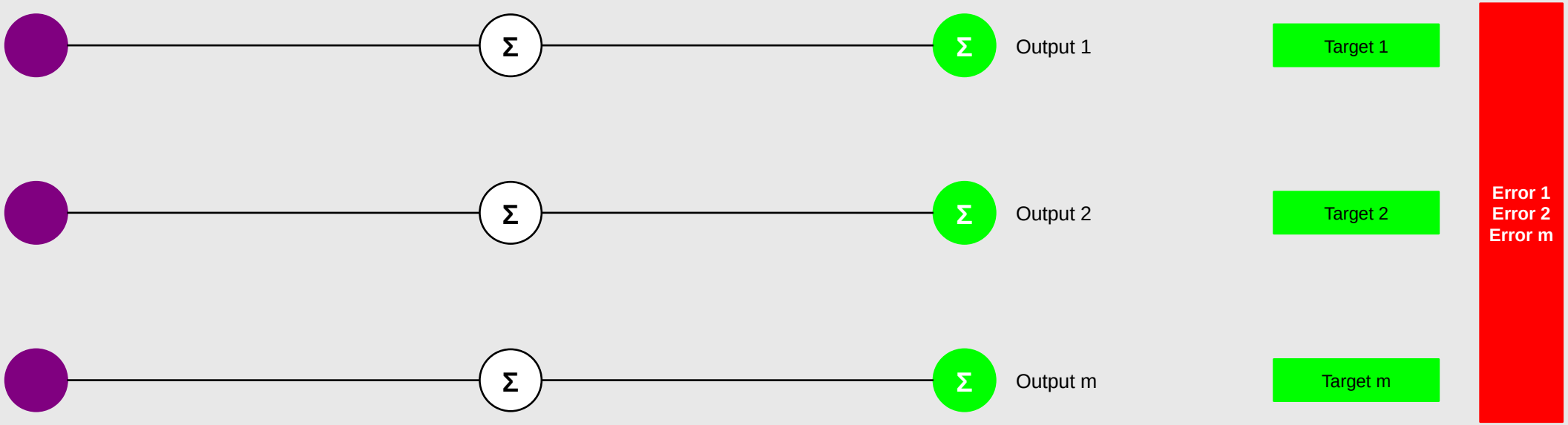
× × ×


Input Data
Output1
Output2
Output3
Error
Class 1
Class 2
Class m
?

At Hidden Layer



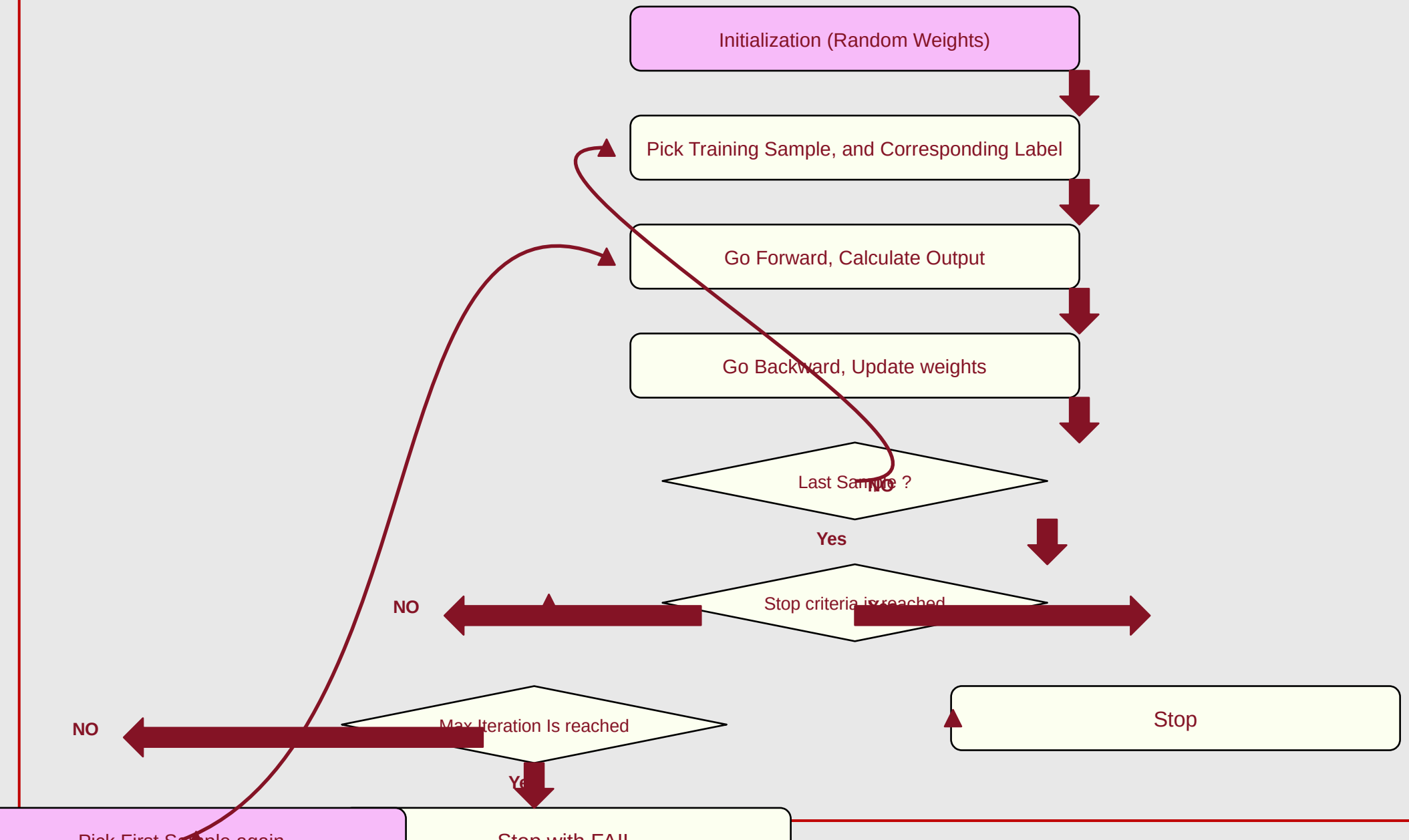
Back Propagation Training Algorithm



Made with draw.io

[1] Initialize all Weights with random values
[2] Provide one training sample at a time (and its corresponding target label)

[3] Go Forward : Start from Input Layer, Calculate Net_j, O_j for each layer till reach output layer


Initialization (Random Weights)
Pick Training Sample, and Corresponding Label
Go Forward, Calculate Output
Go Backward, Update weights
Last Sample ?
NO
Yes
NO
Stop criteria is reached
NO
Max Iteration Is reached
Stop

Stopping Criteria

[1] Total Error = zero (Theoretical)

[2] Total Error < certain value (Threshold)

[3] Percentage Change of Weights is less than certain value (Threshold)

[4] Reach Maximum number of iterations (Epochs) without satisfying [2] or [3]

Stopping is due to reaching Maximum number of iterations = Training is FAILED

Expected Reasons:

[1] The defined Maximum Number of Epochs is small. Try to increase the limit

[2] Under-fitting (Training samples are not adequate). Try to provide more training samples

[3] Model Design is not suitable to training data. Try to add more Hidden Layers or add more Hidden Layers

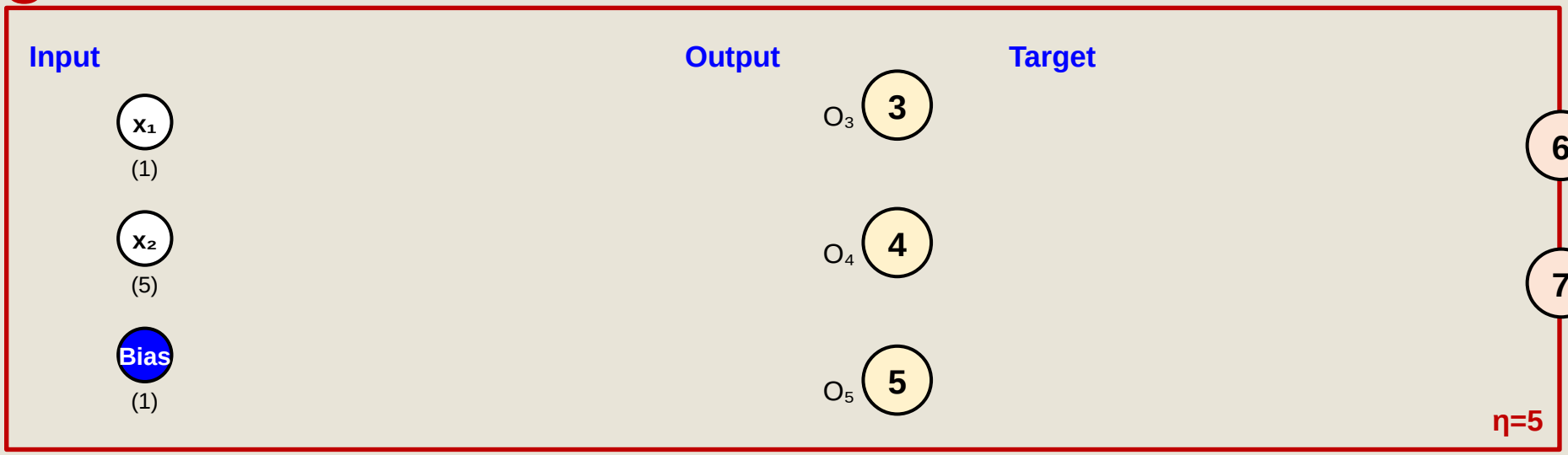
[4] Data is NOT separable by Neural Network . Try Another Models (e.g. SVM with Kernels)

Solved Example

[Update Weights of Output Layer and Hidden Layer]

Using one training sample

Example



Forward

$Net_3 = 1*0.1+5*0.4+1*0.1 = 2.2$

$Net_4 = 1*0.2+5*0.5+1*0.1 = 2.8$

$Net_5 = 1*0.3+5*0.6+1*0.1 = 3.4$

$Net_6 = 0.9*0.3+0.943*0.4+0.968*0.5+1*0.2 = 1.3312$

$Net_7 = 0.9*0.3+0.943*0.4+0.968*0.5+1*0.2 = 1.3312$

$O_3 = 1/(1+e^{-2.2}) = 0.9$

$O_4 = 1/(1+e^{-2.8}) = 0.943$

$O_5 = 1/(1+e^{-3.4}) = 0.968$

$Net_6 = 0.9*0.3+0.943*0.4+0.968*0.5+1*0.2 = 1.3312$

$Net_7 = 0.9*0.3+0.943*0.4+0.968*0.5+1*0.2 = 1.3312$

$O_6 = 1/(1+e^{-1.3312}) = 0.791$

$O_7 = 1/(1+e^{-1.3312}) = 0.791$